

[TA044] BASE DE DATOS – CÁTEDRA MERLINO

Trabajo Práctico – 1.er cuatrimestre de 2026

Optimización de consultas SQL

Base de datos de ventas y pedidos

Integrante	Padrón
Camilo Ignacio Campos Durán	109368
Cristina Machasilla	112594
Carolina Aramayo	106260
Brero Joaquín	110916
Víctor Andre Oliva Montaña	112748

Fecha de entrega: 25 de junio de 2026

Índice

1. Resumen	2
2. Problema y objetivos	2
3. Modelo de datos y conjunto experimental	2
4. Metodología	3
5. Diseño de las optimizaciones	3
5.1. Índices	3
5.2. Reescrituras	3
6. Resultados	4
6.1. Productos más vendidos por fecha	4
6.2. Total vendido por mes	4
6.3. Pedidos de un cliente	4
6.4. Ventas por categoría	4
7. Reproducibilidad y estructura de la entrega	4
8. Conclusiones	5

1. Resumen

Este trabajo estudia el rendimiento de cuatro reportes frecuentes de un sistema de ventas: productos más vendidos en un período, facturación mensual, pedidos de un cliente y ventas por categoría. Se implementó una base PostgreSQL con datos determinísticos y se compararon las consultas originales contra versiones optimizadas usando **EXPLAIN (ANALYZE, BUFFERS)**.

El experimento demuestra que la optimización no consiste solamente en agregar índices. Los índices resultan especialmente efectivos para filtros selectivos, mientras que en consultas que deben recorrer toda la tabla es más importante eliminar joins innecesarios y reducir el conjunto de filas antes de unirlo con otras tablas. La mayor mejora fue de 99,88 % y la menor, de 23,76 %, de acuerdo con la naturaleza de cada reporte.

2. Problema y objetivos

Un comercio registra clientes, productos, categorías, pedidos y sus detalles. Con pocos registros, las consultas analíticas responden de forma aceptable; al crecer la cantidad de pedidos, los recorridos secuenciales y los joins sobre tablas grandes incrementan el tiempo de respuesta.

El objetivo es identificar operaciones costosas mediante planes de ejecución, aplicar índices y reescrituras justificadas, y verificar que las consultas optimizadas preserven exactamente el resultado lógico de las originales.

3. Modelo de datos y conjunto experimental

El modelo posee cinco entidades. Un cliente realiza pedidos; cada pedido contiene uno o más detalles; cada detalle refiere a un producto, que pertenece a una categoría. El detalle conserva el precio unitario de la compra para que los reportes históricos no dependan de cambios futuros en el catálogo.

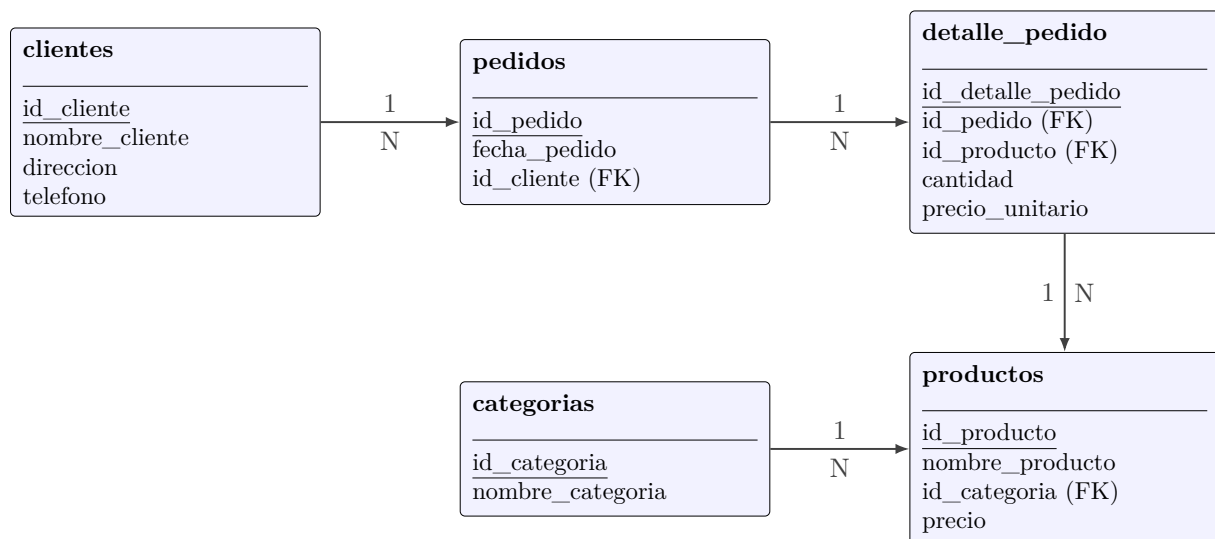


Figura 1: Modelo entidad-relación implementado.

La carga usa **generate_series** para obtener un escenario repetible: 10.000 clientes, 200.000 pedidos distribuidos entre 2022 y 2023, y 2.000.000 de detalles (diez por pedido). Tras la carga y tras la creación de índices se ejecuta **ANALYZE**, de modo que el optimizador disponga de estadísticas actualizadas.

4. Metodología

Las mediciones se realizaron en PostgreSQL 16.14 con `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)`. Para disminuir el efecto de la caché, cada conjunto se ejecutó una vez de calentamiento y una segunda vez para registrar el tiempo. Se observaron tanto el tiempo total como los operadores predominantes, las filas procesadas y los buffers accedidos.

La equivalencia semántica se verifica con `EXCEPT` en ambos sentidos para las cuatro consultas. El script `07-verify-results.sql` aborta si cualquier versión optimizada difiere de la original; por lo tanto, una mejora de rendimiento no se acepta a costa de alterar el resultado.

5. Diseño de las optimizaciones

5.1. Índices

Se crearon índices compuestos alineados con los filtros y joins de los reportes:

Listing 1: Índices relevantes para las consultas selectivas.

```
CREATE INDEX idx_pedidos_fecha_id
ON pedidos (fecha_pedido, id_pedido);

CREATE INDEX idx_pedidos_cliente_id
ON pedidos (id_cliente, id_pedido)
INCLUDE (fecha_pedido);

CREATE INDEX idx_detalle_pedido_cubriendo
ON detalle_pedido (id_pedido, id_producto)
INCLUDE (cantidad, precio_unitario);
```

El primer índice permite localizar los pedidos de un rango temporal. El segundo selecciona los pedidos de un cliente y, al incluir la fecha, habilita un *Index Only Scan*. El tercero cubre el acceso desde pedido a sus detalles sin necesitar visitar el heap cuando el mapa de visibilidad lo permite.

5.2. Reescrituras

En las consultas de productos por fecha y ventas por categoría se agrega primero por `id_producto` y se une luego con las tablas de dimensión. Esta decisión reduce el volumen que participa en los joins finales. En la facturación mensual se eliminó el join con `productos`, ya que no proveía columnas ni filtros para el resultado.

Por ejemplo, la consulta de ventas por categoría reduce primero dos millones de detalles a veinte totales de producto:

Listing 2: Agregación temprana para ventas por categoría.

```
WITH ventas_por_producto AS (
  SELECT id_producto, SUM(precio_unitario * cantidad) AS total_vendido
  FROM detalle_pedido
  GROUP BY id_producto
)
SELECT c.nombre_categoria, SUM(vpp.total_vendido) AS total_vendido
FROM categorias c
JOIN productos p ON p.id_categoria = c.id_categoria
JOIN ventas_por_producto vpp ON vpp.id_producto = p.id_producto
GROUP BY c.id_categoria, c.nombre_categoria
ORDER BY total_vendido DESC;
```

6. Resultados

Consulta	Antes	Después	Mejora
Productos más vendidos por fecha	364,447 ms	61,458 ms	83,14 %
Total vendido por mes	1706,348 ms	1300,889 ms	23,76 %
Pedidos de un cliente	309,181 ms	0,377 ms	99,88 %
Ventas por categoría	1330,589 ms	631,670 ms	52,53 %

Tabla 1: Comparación obtenida con `EXPLAIN ANALYZE`.

6.1. Productos más vendidos por fecha

La versión original recorría las tablas `pedidos` y `detalle_pedido` mediante *Parallel Seq Scan*. El índice por fecha permitió un *Bitmap Index Scan* sobre aproximadamente 8.500 pedidos del mes, y el índice cubriente permitió llegar a sus detalles con *Index Only Scan*. El intervalo semiabierto `[2022-01-01, 2022-02-01)` evita ambigüedades en rangos temporales. La mejora fue de 83,14 %.

6.2. Total vendido por mes

Como el reporte calcula todos los meses y todos los detalles, debe procesar los 2.000.000 de filas de `detalle_pedido`; un índice no puede evitar ese costo. La mejora proviene de remover el join con `productos`, que era lógicamente redundante. El plan conserva el recorrido completo, pero elimina un *Hash Join* adicional y reduce el tiempo en 23,76 %.

6.3. Pedidos de un cliente

El filtro por cliente devuelve veinte pedidos y doscientos detalles: una fracción mínima de las tablas. Antes se recorrían 200.000 pedidos y 2.000.000 de detalles. Después se utilizaron *Index Only Scan* sobre `idx_pedidos_cliente_id` y `idx_detalle_pedido_cubriendo`. El tiempo disminuyó de 309,181 ms a 0,377 ms, una mejora de 99,88 %.

6.4. Ventas por categoría

La consulta original unía cada detalle con producto y categoría antes de agrupar. La nueva versión agrupa primero los detalles por producto: el paso posterior opera sobre veinte filas y cinco categorías. Aunque el escaneo completo inicial sigue siendo inevitable, se reducen los joins posteriores y se obtiene una mejora de 52,53 %.

7. Reproducibilidad y estructura de la entrega

El repositorio incluye scripts separados por etapa, un orquestador y verificaciones automatizadas. Desde `TPG-Base-de-Datos` puede ejecutarse:

Listing 3: Ejecución integral desde `psql`.

```
psql -v ON_ERROR_STOP=1 -U usuario -d base_de_datos \
-f scripts/script-completo.sql
```

El proceso reinicia el esquema, crea las tablas, carga datos, captura planes antes y después, crea índices y comprueba la equivalencia de los resultados. Los planes completos y las mediciones registradas se encuentran en `results/`.

8. Conclusiones

Las consultas correctas desde el punto de vista funcional no son necesariamente eficientes. La evidencia obtenida muestra tres principios: los índices aportan más cuando el predicado es selectivo; un join sin aporte semántico también tiene costo; y agregar antes de unir reduce el trabajo de operadores posteriores.

No todas las consultas se benefician por igual. Cuando la pregunta exige leer toda la tabla, como en la facturación mensual, la mejora posible con índices es limitada. Para escenarios de mayor escala se podrían evaluar particionamiento temporal, vistas materializadas para reportes periódicos y una estrategia de refresco acorde a la frecuencia de actualización.